Senior AI Engineering Curriculum

Day 7: Basic Observability + Week 1 Review

Prometheus, Grafana Dashboards, Latency Metrics & Week 1 Baseline

---

Goal: Every call to your server is now visible, measured, and alertable. Instrument FastAPI with prometheus-fastapi-instrumentator, expose /metrics, scrape with local Prometheus, build a p50/p95 latency + RPS dashboard in Grafana, set an alerting threshold, define TTFT/TBT/E2E latency, log per-request token counts, and baseline your full Week 1 stack throughput before Week 2.

PART A -- Prometheus: Instrumenting Your LLM Server

What is Prometheus and Why Pull-Based Scraping?

Prometheus is an open-source time-series metrics database. It operates on a **pull model**: instead of your application pushing metrics somewhere, Prometheus scrapes a /metrics HTTP endpoint on your service every 15 seconds. This makes it trivial to add observability without changing your deployment topology -- just expose an endpoint and add a scrape target. For LLM serving, Prometheus stores every latency histogram bucket, request counter, and GPU gauge reading, making them queryable at any time via PromQL.

The Four Prometheus Metric Types

Type	Behaviour	LLM Use Case	Key PromQL
Counter	Monotonically increasing, never resets	Total requests, total tokens	rate(), increase()
Gauge	Current snapshot value, up or down	VRAM used, queue depth, GPU util	avg_over_time()
Histogram	Counts observations in configurable buckets	Request latency, TTFT, TBT	histogram_quantile()
Summary	Pre-computed quantiles per process	Avoid -- use Histogram instead	Direct label access

Rule: Always use Histogram for latency, never Summary. Histograms aggregate correctly across multiple server replicas; Summaries do not.

Practical Implementation -- Prometheus

A.1 Install and Auto-Instrument FastAPI

```
pip install prometheus-fastapi-instrumentator prometheus-client
```

```
# server.py -- add two lines to get HTTP metrics automatically
from fastapi import FastAPI
from prometheus_fastapi_instrumentator import Instrumentator
from prometheus_client import Counter, Histogram, Gauge
import torch, time, asyncio
```

```
app = FastAPI(title="LLM Inference API")

# Two lines: instrument all routes, expose /metrics endpoint
Instrumentator().instrument(app).expose(app)
# This automatically creates:
#   http_requests_total{method, status, handler}          -- Counter
#   http_request_duration_seconds{method, status, handler} -- Histogram

# Custom LLM-specific metrics
PROMPT_TOKENS = Counter(
    "llm_prompt_tokens_total", "Total prompt tokens processed", ["model"])
COMPLETION_TOKENS = Counter(
    "llm_completion_tokens_total", "Total completion tokens generated", ["model"])
GEN_LATENCY = Histogram(
    "llm_generation_latency_seconds", "End-to-end generation latency",
    ["model"], buckets=[0.1, 0.5, 1.0, 2.0, 5.0, 10.0, 20.0, 30.0, 60.0])
TTFT_HIST = Histogram(
    "llm_ttft_seconds", "Time-to-first-token latency",
    ["model"], buckets=[0.05, 0.1, 0.25, 0.5, 1.0, 2.0, 5.0])
VRAM_USED = Gauge(
    "llm_vram_used_bytes", "GPU VRAM currently allocated", ["gpu"])
QUEUE_DEPTH = Gauge(
    "llm_queue_depth", "Requests currently in queue")
MODEL_LOAD_SECS = Gauge(
    "llm_model_load_seconds", "Time to load model at startup", ["model"])

async def poll_vram():
    while True:
        if torch.cuda.is_available():
            for i in range(torch.cuda.device_count()):
                VRAM_USED.labels(gpu=str(i)).set(torch.cuda.memory_allocated(i))
        await asyncio.sleep(15)

@app.on_event("startup")
async def startup():
    asyncio.create_task(poll_vram())
```

A.2 Recording TTFT and Token Counts Per Request

```
from pydantic import BaseModel, Field
from transformers import AutoTokenizer, AutoModelForCausalLM, TextIteratorStreamer
import threading

MODEL_ID  = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
model     = AutoModelForCausalLM.from_pretrained(MODEL_ID).eval()

class GenerateRequest(BaseModel):
    prompt:         str   = Field(..., min_length=1)
    max_new_tokens: int   = Field(200, ge=1, le=2048)
    temperature:    float = Field(0.7, ge=0.0, le=2.0)

@app.post("/generate")
async def generate(req: GenerateRequest):
    QUEUE_DEPTH.inc()
    t_start = time.perf_counter()
```

```
    try:
        inputs    = tokenizer(req.prompt, return_tensors="pt")
        in_toks   = inputs["input_ids"].shape[1]
        streamer  = TextIteratorStreamer(tokenizer, skip_prompt=True,
                                         skip_special_tokens=True)
        thread    = threading.Thread(target=model.generate, kwargs=dict(
            **inputs, max_new_tokens=req.max_new_tokens, streamer=streamer,
            do_sample=req.temperature > 0, temperature=req.temperature,
            pad_token_id=tokenizer.eos_token_id))
        thread.start()

        full_text, ttft = "", None
        for tok in streamer:
            if ttft is None:
                ttft = time.perf_counter() - t_start
                TTFT_HIST.labels(model=MODEL_ID).observe(ttft)
            full_text += tok
        thread.join()

        elapsed  = time.perf_counter() - t_start
        out_toks = len(tokenizer.encode(full_text))

        GEN_LATENCY.labels(model=MODEL_ID).observe(elapsed)
        PROMPT_TOKENS.labels(model=MODEL_ID).inc(in_toks)
        COMPLETION_TOKENS.labels(model=MODEL_ID).inc(out_toks)

        return {"text": full_text, "input_tokens": in_toks,
                "output_tokens": out_toks, "elapsed_s": round(elapsed,3),
                "ttft_s": round(ttft,3) if ttft else None}
    finally:
        QUEUE_DEPTH.dec()
```

A.3 Prometheus Scrape Config + Docker Compose

```
# prometheus.yml
global:
  scrape_interval:     15s
  evaluation_interval: 15s

rule_files:
  - "alert_rules.yml"

scrape_configs:
  - job_name: "llm-server"
    static_configs:
      - targets: ["llm-server:8080"]   # container name in Docker network

  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "node"
    static_configs:
      - targets: ["node-exporter:9100"]
```

```
# docker-compose.monitoring.yml
version: "3.8"
```

```yaml
services:
  prometheus:
    image: prom/prometheus:v2.50.0
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./alert_rules.yml:/etc/prometheus/alert_rules.yml:ro
      - prometheus_data:/prometheus
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
      - "--storage.tsdb.retention.time=15d"
    restart: unless-stopped

  node-exporter:
    image: prom/node-exporter:v1.7.0
    ports:
      - "9100:9100"
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
    command:
      - "--path.procfs=/host/proc"
      - "--path.sysfs=/host/sys"
    restart: unless-stopped

volumes:
  prometheus_data:

# Start:   docker compose -f docker-compose.monitoring.yml up -d
# Test:    curl http://localhost:9090/api/v1/targets
# Metrics: curl http://localhost:8080/metrics | grep llm_
```

A.4 Alert Rules

```yaml
# alert_rules.yml
groups:
  - name: llm_alerts
    rules:

      - alert: HighP95Latency
        expr: |
          histogram_quantile(0.95,
            sum by (le) (rate(llm_generation_latency_seconds_bucket[5m]))
          ) > 10
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "p95 latency above 10s (current: {{ $value | humanizeDuration }})"

      - alert: HighErrorRate
        expr: |
          sum(rate(http_requests_total{status=~"5.."}[5m]))
          / sum(rate(http_requests_total[5m])) > 0.05
        for: 1m
```

```
    labels:
      severity: critical
    annotations:
      summary: "Error rate above 5% (current: {{ $value | humanizePercentage }})"

  - alert: LLMServerDown
    expr: up{job="llm-server"} == 0
    for: 1m
    labels:
      severity: critical
    annotations:
      summary: "LLM server is unreachable"

  - alert: HighVRAM
    expr: (llm_vram_used_bytes / 8e10) > 0.90
    for: 3m
    labels:
      severity: warning
    annotations:
      summary: "GPU VRAM above 90%"
```

PART B -- Grafana Dashboards

B.1 Add Grafana to the Monitoring Stack

```
# Add to docker-compose.monitoring.yml
  grafana:
    image: grafana/grafana:10.2.0
    ports:
      - "3000:3000"
    environment:
      - GF_AUTH_ANONYMOUS_ENABLED=true
      - GF_AUTH_ANONYMOUS_ORG_ROLE=Admin
    volumes:
      - grafana_data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning
    depends_on: [prometheus]
    restart: unless-stopped

# grafana/provisioning/datasources/prometheus.yaml
apiVersion: 1
datasources:
  - name: Prometheus
    type: prometheus
    url: http://prometheus:9090
    isDefault: true
    access: proxy

# Grafana UI: http://localhost:3000
```

B.2 Core PromQL Queries for LLM Dashboard

```
# Panel 1: Requests Per Second (RPS)
sum(rate(http_requests_total{job="llm-server"}[1m]))

# By HTTP status
sum by (status) (rate(http_requests_total{job="llm-server"}[1m]))

# Panel 2: E2E Latency -- p50, p95, p99
histogram_quantile(0.50, sum by (le)(rate(llm_generation_latency_seconds_bucket[5m])))
histogram_quantile(0.95, sum by (le)(rate(llm_generation_latency_seconds_bucket[5m])))
histogram_quantile(0.99, sum by (le)(rate(llm_generation_latency_seconds_bucket[5m])))

# Panel 3: TTFT -- p50, p95
histogram_quantile(0.50, sum by (le)(rate(llm_ttft_seconds_bucket[5m])))
histogram_quantile(0.95, sum by (le)(rate(llm_ttft_seconds_bucket[5m])))

# Panel 4: Token Throughput (tok/s)
rate(llm_completion_tokens_total[1m])    # generation tokens/sec

# Panel 5: Error rate
sum(rate(http_requests_total{status=~"5.."}[5m]))
/ sum(rate(http_requests_total[5m]))

# Panel 6: VRAM used (GB)
llm_vram_used_bytes / 1e9
```

```
# Panel 7: Queue depth
llm_queue_depth

# Panel 8: Host CPU usage
100 - (avg by(instance)(rate(node_cpu_seconds_total{mode="idle"}[1m])) * 100)
```

B.3 Dashboard JSON Skeleton (Grafana 10)

```
# Paste into Grafana -> Dashboards -> Import -> Paste JSON
{
  "title": "LLM Inference Server - Week 1",
  "panels": [
    {
      "title": "Requests Per Second",
      "type": "timeseries",
      "targets": [{
        "expr": "sum(rate(http_requests_total{job='llm-server'}[1m]))",
        "legendFormat": "RPS"
      }]
    },
    {
      "title": "Generation Latency",
      "type": "timeseries",
      "targets": [
        {"expr": "histogram_quantile(0.50, sum by (le)(rate(llm_generation_latency_seconds_bucket[5m])))", "lege
        {"expr": "histogram_quantile(0.95, sum by (le)(rate(llm_generation_latency_seconds_bucket[5m])))", "lege
        {"expr": "histogram_quantile(0.99, sum by (le)(rate(llm_generation_latency_seconds_bucket[5m])))", "lege
      ]
    },
    {
      "title": "TTFT",
      "type": "timeseries",
      "targets": [
        {"expr": "histogram_quantile(0.50, sum by (le)(rate(llm_ttft_seconds_bucket[5m])))", "legendFormat": "TT
        {"expr": "histogram_quantile(0.95, sum by (le)(rate(llm_ttft_seconds_bucket[5m])))", "legendFormat": "TT
      ]
    },
    {
      "title": "Token Throughput",
      "type": "stat",
      "targets": [{"expr": "rate(llm_completion_tokens_total[1m])", "legendFormat": "tok/s"}]
    },
    {
      "title": "VRAM Used (GB)",
      "type": "gauge",
      "targets": [{"expr": "llm_vram_used_bytes / 1e9", "legendFormat": "GPU 0"}],
      "fieldConfig": {"defaults": {"max": 80, "unit": "decgbytes"}}
    }
  ],
  "refresh": "10s",
  "time": {"from": "now-30m", "to": "now"}
}
```

PART C -- Latency Definitions & Throughput Baseline

The Three LLM Latency Metrics

Metric	Full Name	Formula	Bottleneck	Target
TTFT	Time-to-First-Token	t(first_output_token) - t(request_sent)	Prefill: prompt length, GPU speed	<1s GPU / <5s CPU
TBT	Time-Between-Tokens	1 / generation_tok_per_s * 1000 (ms)	Decode: model size, bandwidth	<100ms (>10 tok/s)
E2E	End-to-End Latency	TTFT + (out_tokens x TBT)	Both combined	Workload dependent

Track all three independently. High TTFT with low TBT = prefill bottleneck (shorten prompt, use Flash Attention). Low TTFT with high TBT = decode bottleneck (quantize, use faster GPU). E2E alone cannot distinguish these two cases.

C.1 Comprehensive Latency Measurement Script

```
import time, statistics, threading
from transformers import AutoTokenizer, AutoModelForCausalLM, TextIteratorStreamer
import torch

MODEL_ID  = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
model     = AutoModelForCausalLM.from_pretrained(MODEL_ID).eval()

def measure_latencies(prompt: str, max_new_tokens: int = 80) -> dict:
    inputs  = tokenizer(prompt, return_tensors="pt")
    in_toks = inputs["input_ids"].shape[1]
    streamer = TextIteratorStreamer(tokenizer, skip_prompt=True,
                                    skip_special_tokens=True)
    thread   = threading.Thread(target=model.generate, kwargs=dict(
        **inputs, max_new_tokens=max_new_tokens, streamer=streamer,
        do_sample=False, pad_token_id=tokenizer.eos_token_id))

    t0 = time.perf_counter()
    thread.start()

    ttft, token_intervals, full_text = None, [], ""
    prev_t = None
    for tok in streamer:
        now = time.perf_counter()
        if ttft is None:
            ttft   = now - t0
            prev_t = now
        else:
            token_intervals.append(now - prev_t)
            prev_t = now
        full_text += tok
    thread.join()

    e2e      = time.perf_counter() - t0
    out_toks = len(tokenizer.encode(full_text))
    avg_tbt  = statistics.mean(token_intervals) if token_intervals else 0
```

```
    return {
        "ttft_s"     : round(ttft, 4) if ttft else 0,
        "avg_tbt_ms" : round(avg_tbt * 1000, 2),
        "e2e_s"      : round(e2e, 4),
        "in_tokens"  : in_toks,
        "out_tokens" : out_toks,
        "tok_per_s"  : round(out_toks / e2e, 1) if e2e else 0,
    }

PROMPT = "Explain the transformer attention mechanism in detail."
print(f"{'Run':<5} {'TTFT':>8} {'TBT ms':>9} {'E2E':>7} {'tok/s':>8}")
print("-" * 42)
results = []
for i in range(5):
    m = measure_latencies(PROMPT)
    results.append(m)
    print(f"{i+1:<5} {m['ttft_s']:>8.3f}s {m['avg_tbt_ms']:>8.1f}ms "
          f"{m['e2e_s']:>6.2f}s {m['tok_per_s']:>8.1f}")

ttfts = [r["ttft_s"] for r in results]
e2es  = [r["e2e_s"]  for r in results]
print(f"
Mean TTFT: {statistics.mean(ttfts):.3f}s")
print(f"Mean E2E : {statistics.mean(e2es):.2f}s")
print(f"Mean tok/s: {statistics.mean(r['tok_per_s'] for r in results):.1f}")
```

C.2 Throughput Baseline Script

```
# baseline_throughput.py -- measure Week 1 naive stack ceiling
# Run this now; compare against vLLM results in Week 2

import asyncio, httpx, time, statistics

BASE_URL  = "http://localhost:8080"
PROMPTS   = [
    "What is gradient descent?",
    "Explain transformer attention.",
    "Define backpropagation.",
    "What is a neural network?",
    "Explain softmax.",
] * 4    # 20 total

async def bench(concurrency: int) -> dict:
    latencies, tokens, errors = [], [], 0
    sem = asyncio.Semaphore(concurrency)

    async def req(prompt):
        nonlocal errors
        async with sem:
            t0 = time.perf_counter()
            try:
                async with httpx.AsyncClient(timeout=120) as c:
                    r = await c.post(f"{BASE_URL}/generate",
                                     json={"prompt": prompt, "max_new_tokens": 80,
                                           "temperature": 0})
                    r.raise_for_status()
```

```
                    latencies.append(time.perf_counter() - t0)
                    tokens.append(r.json().get("output_tokens", 0))
            except Exception:
                errors += 1

    t_start = time.perf_counter()
    await asyncio.gather(*[req(p) for p in PROMPTS])
    wall = time.perf_counter() - t_start

    lat_s = sorted(latencies)
    n     = len(lat_s)
    return {
        "concurrency": concurrency,
        "wall_s"     : round(wall, 2),
        "rps"        : round(len(PROMPTS) / wall, 2),
        "tok_s"      : round(sum(tokens) / wall, 1),
        "p50_s"      : round(lat_s[n//2], 2) if lat_s else 0,
        "p95_s"      : round(lat_s[int(n*0.95)], 2) if lat_s else 0,
        "errors"     : errors,
    }

async def run():
    print("Week 1 Throughput Baseline (naive HuggingFace)")
    print(f"{'Conc':>6} {'RPS':>7} {'tok/s':>8} {'p50':>8} {'p95':>8} {'Err':>5}")
    print("-" * 47)
    for c in [1, 2, 4, 8]:
        r = await bench(c)
        print(f"{r['concurrency']:>6} {r['rps']:>7.2f} {r['tok_s']:>8.1f} "
              f"{r['p50_s']:>7.2f}s {r['p95_s']:>7.2f}s {r['errors']:>5}")
    print("
Save this output -- vLLM (Day 9) should show 5-10x improvement.")

asyncio.run(run())
```

PART D -- Week 1 Self-Assessment & Review

Week 1 wrap-up: Before moving to Week 2 advanced topics, verify you can answer and demonstrate each item below. Any gap is a topic to revisit -- Week 2 builds directly on these foundations.

Day-by-Day Verification Checklist

Day	Topic	Can you demonstrate?	Verification
1A	HuggingFace pipeline	Load gpt2 and generate 50 tokens	pipeline('text-generation', model='gpt2')
1A	AutoModel API	Generate with do_sample=False, count tokens	AutoModelForCausalLM.from_pretrained()
1B	Ollama CLI + REST	Pull llama3, run via CLI, curl /api/generate	ollama run llama3 | curl localhost:11434
1B	OpenAI compat SDK	Point openai SDK at Ollama localhost:11434	OpenAI(base_url='...localhost:11434/v1')
2	TTFT streaming	Measure TTFT with TextIteratorStreamer	First token time recorded in loop
2	5 model formats	Name GGUF, SafeTensors, AWQ, GPTQ, ONNX + use case	See format compatibility table
3	GGUF conversion	SafeTensors -> GGUF Q4_K_M via llama.cpp	convert_hf_to_gguf.py + llama-quantize
3	VRAM formula	Calculate VRAM for any model + dtype + KV	params_B x bytes x 1.2 + kv_cache
3	CPU vs GPU bench	Compare same prompt on CPU and CUDA, tok/s	Benchmarks model.to('cpu') vs model.to('cuda')
4	AWQ INT4	Quantize phi-2 to INT4, save, reload, infer	AutoAWQForCausalLM.quantize()
4	KV cache speedup	Benchmark use_cache=True vs False	model.generate(use_cache=False/True)
5	FastAPI /generate	Serve model with Pydantic validation at :8080	uvicorn server:app --port 8080
5	Token truncation	Truncate prompt to fit context window	tokenizer.encode() + max_length
5	Few-shot JSON	Extract JSON with 2 examples in history	messages=[examples...]+query
6	Multi-stage Docker	Build lean image, run with --gpus all, volume	docker build --target runtime
6	Image versioning	Tag with model+quant+version, rollback steps	llama3-8b:fp16-v1.0.2 tag convention
7	Prometheus /metrics	Expose metrics, scrape locally, see in UI	Instrumentator().instrument(app)
7	Grafana p50/p95	Build latency panel with histogram_quantile()	histogram_quantile(0.95, ...)
7	Throughput baseline	Record tok/s, RPS, p50/p95 for your stack	baseline_throughput.py output saved

Week 1 Architecture Summary

```
# What you own after Week 1 -- the complete local-to-containerised stack:
#
# [OBSERVABILITY]   Prometheus /metrics  ->  Grafana p50/p95/RPS dashboard
#                   TTFT + TBT + E2E baselines recorded
#        |
# [CONTAINERS]      Multi-stage Dockerfile (builder + runtime stages)
#                   Volume-mounted weights (never bake into image)
#                   /health + /ready + /version endpoints
#                   Model versioning: repo/model:quant-version tags
```

```
#        |
# [SERVING]         FastAPI + Pydantic validation + streaming (SSE)
#                   Token counting + context window truncation
#                   Prompt engineering: few-shot, CoT, temperature, top-p
#        |
# [QUANTIZATION]    AWQ INT4 (best GPU quality, 4x VRAM reduction)
#                   bitsandbytes NF4 (zero-calibration, slower runtime)
#                   GGUF Q4_K_M (CPU + consumer GPU, Ollama native)
#                   KV cache: prefill vs decode, VRAM growth formula
#        |
# [MODEL FORMATS]   SafeTensors (HF default), GGUF (Ollama/llama.cpp)
#                   AWQ, GPTQ, EXL2, ONNX -- when to use each
#        |
# [RUNTIMES]        HuggingFace Transformers (pipeline / AutoModel)
#                   Ollama (llama.cpp backend, OpenAI-compatible REST API)
#                   Streaming: TextStreamer (blocking), TextIteratorStreamer
#        |
# [HARDWARE]        VRAM formula: params x dtype_bytes x 1.2 + KV cache
#                   GPU tier guide: RTX3060 -> H100 -> Apple M2 Ultra
#                   CPU vs GPU: NVLink vs PCIe, bandwidth-bound decode
#
# Week 2 replaces the HuggingFace serving layer with vLLM + PagedAttention
# and scales from 1 user to 100+ concurrent users.
```

Testing & Metrics

Acceptance Criteria Before Moving to Week 2

Check	How to verify	Pass condition
Prometheus scraping /metrics	curl localhost:8080/metrics | grep llm_	llm_ metrics present
Counters increment on requests	fire 5 requests, query Prometheus counter	http_requests_total += 5
p95 latency visible in Grafana	histogram_quantile(0.95,...) panel loaded	Shows numeric value
Alert rule evaluates correctly	prometheus/alerts page in Prometheus UI	Alert shown as pending/firing
TTFT histogram populated	curl /metrics | grep llm_ttft_seconds	Bucket counts > 0
Throughput baseline saved	baseline_throughput.py output saved	tok/s and latencies recorded
All Day 1-7 checklist items done	Manual demonstration	Each item runnable from memory

Key Takeaways

• **Two lines to instrument FastAPI**: Instrumentator().instrument(app).expose(app) gives HTTP request rate, latency histograms, and error counts across all routes automatically.
• **Always Histogram for latency, never Summary.** Histograms aggregate across replicas; Summaries are per-process only.
• **TTFT, TBT, and E2E are three independent signals** that diagnose different bottlenecks. Track all three from the start -- retrofitting them later is painful.

• **Alert on p95, not mean.** A 10-second p99 is invisible in a 1-second mean if 99% of requests are fast. Set alert thresholds at percentiles.
• **Record your Week 1 throughput baseline now.** Naive HuggingFace at concurrency=1 typically delivers 5-30 tok/s on a GPU. After vLLM (Day 9) the same hardware should reach 500-2000 tok/s. The baseline makes the Week 2 improvements concrete and measurable.

Week 1 complete. Next: **Day 8 (Week 2 Day 1) -- Evaluation & Production Readiness**: write 20 golden test cases, LLM-as-judge scoring, pytest CI integration, and regression gating on every model/quantisation change.